

IndieGen Asset Studio

Track 1 · Multimodal Content Creation Tools

A full-chain, multimodal AI asset generation platform for indie game developers — from one text setting to concept art, depth, PBR materials, music, ambient SFX, sprite sheets and a 3D character, with native AMD Radeon / ROCm support.

[Project Profile \(PDF\)](#) · [Supplementary deck](#) · [Demo video](#) · [Live ROCm benchmark](#)

Track 1 · Multimodal Content Creation Tools

A full-chain, multimodal AI asset generation platform for indie game developers — from one text setting to concept art, depth, PBR materials, music, ambient SFX, sprite sheets and a 3D character, with native AMD Radeon / ROCm support.

1. Project Background

Indie teams spend 40-70 % of their budget on outsourced art, music and sound effects — and outsourcing to multiple vendors makes style consistency almost

impossible to control. A single environment or character is re-designed by different artists at different stages, and the results drift apart: the concept art, the in-game sprite, the 3D model, and the audio no longer feel like they belong to the same world.

IndieGen Asset Studio attacks both problems at once:

1. One chain instead of many vendors. A single text setting (e.g. "post-apocalyptic wasteland bar") drives a linear pipeline — concept art → depth map → PBR materials → music → SFX → sprite sheet → 3D model — all conditioned on the same reference image and prompt, so style stays consistent across every asset.
2. One GPU instead of a cloud bill. The whole chain runs locally on a commodity AMD Radeon GPU through ROCm, so small studios keep iteration fast, private and free.

The project is built as a product, not just a set of scripts: a browser UI, a FastAPI backend with a job queue and SQLite archive, ComfyUI workflows W1-W7, custom ComfyUI nodes, and an optional Three.js scene viewer that assembles a batch into an interactive 3D environment with an automated character performance.

2. Target Users & Application Scenarios

Target users

User group	How they use it
Solo indie developers	Replace outsourced concept art / sprite / audio with local one-click generation
Small studios (2-10 people)	Pre-production concepting, style exploration, asset prototyping before hiring specialists
Game jam teams	Generate a complete, coherent asset set for a 48-72 h jam in under an hour
Content creators / streamers	Build themed game-art packages (environments + characters + music + SFX)
AI tooling enthusiasts	Extend the ComfyUI workflow graph with their own LoRAs and nodes

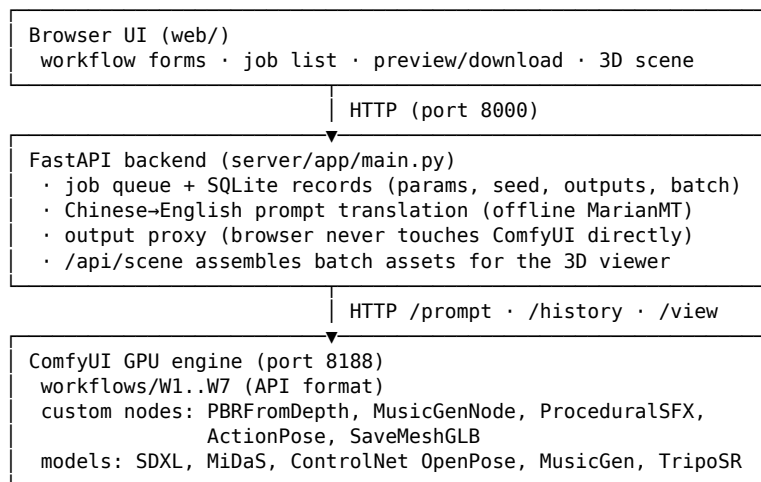
Application scenarios

- Scenario 1 — Wasteland Bar demo (the flagship scenario). A developer types "post-apocalyptic wasteland bar, neon signs, metal counter, dust" and receives: a cinematic 1024x1024 concept image, its depth map, four PBR texture maps, a dark industrial background music track, glass-clink and bar ambience SFX, an 8-frame running sprite atlas with JSON config, and (via W7) a 3D character model — all belonging to the same world. The scene viewer then places the generated assets in an interactive 3D bar where the bartender character walks in, drinks, toasts, wipes the counter and listens to the generated music, triggered by the generated SFX.
- Scenario 2 — Character pipeline for 2D action games. From one character

sheet, W6 generates run / attack sprite atlases in engine-ready format (PNG grid + JSON frame config), keeping character identity consistent across animations.

- Scenario 3 — Environment concepting for 3D scenes. W1 + W2 + W3 turn a text setting into a depth-displaced, PBR-shaded environment background that can be dropped into Blender/Unity.
- Scenario 4 — Audio prototyping. W4/W5 generate looping background music and trigger SFX from text/type selection, letting designers iterate on mood without a composer.

3. System Architecture



Design decisions:

- API-format workflow JSON (workflows/) is submitted directly to ComfyUI's `/prompt` endpoint; UI-format JSON is never used server-side.
- Batch-based archiving — every job writes to `ComfyUI/output/<batch>/<workflow>_<readable>.png/mp3`, and the asset browser lets users reuse server-side outputs as inputs, closing the loop `W1 → W2 → W3 → W6 → W7`.
- Human-in-the-loop — downstream stages run only after the user approves the previous stage, which the web UI supports naturally.
- Reproducibility — prompt, negative prompt, seed, steps, CFG and model names are stored per job in SQLite and displayed in the UI.

4. Model & Algorithm Introduction

Stage	Method / model	Algorithm notes
W1 concept art	Stable Diffusion XL (SDXL base 1.0)	Text prompt → VAE-encoded latent → KSampler (Euler, normal schedule, fixed seed) → VAE decode; negative prompt + CFG 7 for quality; 1024x1024 output
W2 depth map	MiDaS DPT-Hybrid	Monocular depth estimation via a dense prediction transformer (controlnet_aux preprocessor), 512px output used as geometry source
W3 PBR materials	Custom PBRFromDepth	Numerical gradients of the depth map → tangent-space normals; height = depth; roughness from gradient magnitude; metalness as a tunable constant (0-1)
W4 background music	MusicGen-small (transformers)	Autoregressive audio generation conditioned on a text prompt; ~50 tokens/s codec rate; temperature 1.0 sampling with seed; 44.1 kHz output
W5 ambient SFX	Custom ProceduralSFX	Physically-inspired DSP: glass clinks = damped multi-sinusoid bursts (2.2-8.2 kHz); murmur = band-passed noise with syllable envelopes; bar ambience = rumble + murmur + clinks mix
W6 sprite sheet	Custom ActionPose + ControlNet OpenPose (SDXL)	Procedural 18-keypoint skeletons for run/attack cycles → OpenPose conditioning → per-frame SDXL img2img (denoise 0.45-0.55, strength 0.85) with the character sheet as anchor → ImageGrid atlas + JSON config
W7 2D→3D	TripoSr	Single-image large-reconstruction-model: image → sparse-view triplane → 3D mesh; exported as GLB (Y-up) + OBJ with a custom save node
Prompt translation	Helsinki-NLP opus-mt-zh-en (MarianMT)	Offline Chinese→English at submit time; English passes through unchanged
Scene viewer	Three.js	Concept art as a depth-displaced poster (W2 displacement + W3 normal), procedural bartender rig, scripted performance loop, generated music/SFX playback

Consistency strategy

The pipeline treats the ****character sheet / concept image** as a conditioning anchor**: W2-W7 all consume the same image, and W6 further constrains each frame with the same OpenPose skeleton family, fixed seed, and identical LoRA, so the generated sprites do not drift between frames or vendors.

5. AMD Radeon GPU / ROCm Adaptation

Adaptation approach

- scripts/setup_comfyui.sh auto-detects the GPU: NVIDIA → CUDA 12.8 wheels, AMD (/dev/kfd or rocminfo) → ROCm 7.1 wheels, otherwise CPU. The

three branches pin matching torch/torchvision/torchaudio versions to prevent ABI mismatch.

- PyTorch's ROCm build exposes the standard cuda API, so ComfyUI, transformers, ControlNet and all custom nodes run unchanged — no HIP-specific code is needed in the project.
- Verified live on AMD Radeon gfx1100 (Navi 31), 48 GiB VRAM, with torch 2.11.0+rocm7.1 and ComfyUI 0.30.0; ComfyUI reports
cuda:0 AMD Radeon Graphics : native.

Measured performance (live run, benchmark script)

Workflow	Configuration	GPU exec	Wall time	GPU max	VRAM peak	Power peak
W1 concept art	SDXL 1024 ² , 25 steps	14.6 s	15.3 s	100 %	9.6 GB	270 W
W2 depth	MiDaS 512	1.1 s	7.7 s	93 %	12.2 GB	115 W
W3 PBR	4 maps	1.0 s	1.6 s	50 %	12.2 GB	57 W
W4 music	MusicGen-small 8 s	9.6 s	10.7 s	100 %	12.6 GB	124 W
W5 SFX	glass clink 4 s	0.04 s	1.5 s	0 %	12.4 GB	63 W
W6 sprite	8 frames, 20 steps	20.9 s	21.3 s	100 %	21.7 GB	271 W

Full chain (W1-W6, one of each): ~57 s of GPU execution time. Reproduce with `scripts/benchmark_amd.py`. Details in `docs/AMD_ROCm_EN.md`.

6. Demonstration Scenario — "The Wasteland Bartender"

The flagship demo follows the project's main character — a post-apocalyptic wasteland bartender (goggles, worn leather jacket, cel-shaded game sprite) — through the entire chain:

1. Text → W1 concept art. The bar interior is generated at 1024x1024 in ~15 s from a single text setting.
2. Character sheet → W6 sprite atlas. The bartender sheet plus the run action produces an 8-frame atlas + JSON in ~21 s, directly importable into a game engine.
3. Depth + materials. W2/W3 derive the depth map and four PBR maps that give the environment real geometry.
4. Music + SFX. W4 produces a dark industrial electronic loop; W5 produces glass clinks and bar ambience.
5. 3D scene. The batch is assembled into an interactive Three.js bar where the bartender performs an automated loop (walk in → drink → toast → wipe the counter), with generated music and SFX playing.

7. Innovation & Practical Value

- Full-chain style lock: one conditioning anchor across seven asset types, eliminating cross-vendor style drift.
- Engine-ready output: PNG atlas + JSON frame config, PBR maps, GLB/OBJ — no post-processing required before import.

- Low-cost local GPU: entire chain on one Radeon card (peak VRAM 21.7 GB; offloading supports 8-16 GB cards), no per-call cloud fees.
- Bilingual UX: Chinese prompts auto-translate offline to English for SDXL / MusicGen.
- Reproducible & auditable: every job's full parameter set is stored and displayed; fixed seeds make results repeatable.
- Product-grade packaging: docker-compose + nginx + systemd deployment, idempotent host script, proxy-only networking (8188 never exposed).

8. Roadmap

Phase	Status	Content
P0	□	Repository + full design doc
P1	□	ComfyUI install, W1/W2 working
P2	□	Web backend + UI (submit/progress/preview)
P3	□	W3 materials + W4 music + W5 SFX on the web
P4	□	W6 sprite atlas + JSON config
P5	□	Deployment suite (docker/nginx/systemd)
P6	□	W7 hardening, AnimateDiff frame interpolation, per-character LoRA training, WebRTC live preview

9. Appendix — Repository Layout

```

.
├── README.md / README_EN.md      # documentation (CN + EN)
├── scripts/                       # setup · model download · deploy · benchmark
├── docs/                         # model manifest · AMD/ROCm notes · this profile
├── workflows/                   # ComfyUI API workflows W1-W7
├── nodes/indie_studio_nodes/     # 5 custom ComfyUI nodes
├── server/                      # FastAPI backend + SQLite (jobs.db)
├── web/                         # browser UI + Three.js scene viewer
├── deploy/                      # docker-compose · nginx · systemd
├── assets/demo/                 # generated "wasteland bar" demo batch
└── deliverables/                # PDF · PPT · demo video · benchmark data

```

All documentation, slides, and the demo video for this submission are in English.